Actividad 27 – Despliegue web y Docker

Proyecto: **Eventia**

Universidad Rafael Landívar
Curso: Programación Web
Fecha: 29 de octubre de 2026

Integrantes
Carlos Hugo Escobar
Jordin García

1. URL pública y repositorio

- URL pública: https://eventia-inrv.onrender.com
- Health check: https://eventia-inrv.onrender.com/api/health → `{"status":"ok"}`
- Repositorio: https://github.com/carloshugoeg/Eventia
- Pull request: https://github.com/carloshugoeg/Eventia/pull/4 (rama `actividad-27-docker`)

2. Evidencia de despliegue (Render)

Plataforma: Render, Web Service de tipo Docker, plan free, región Oregon.

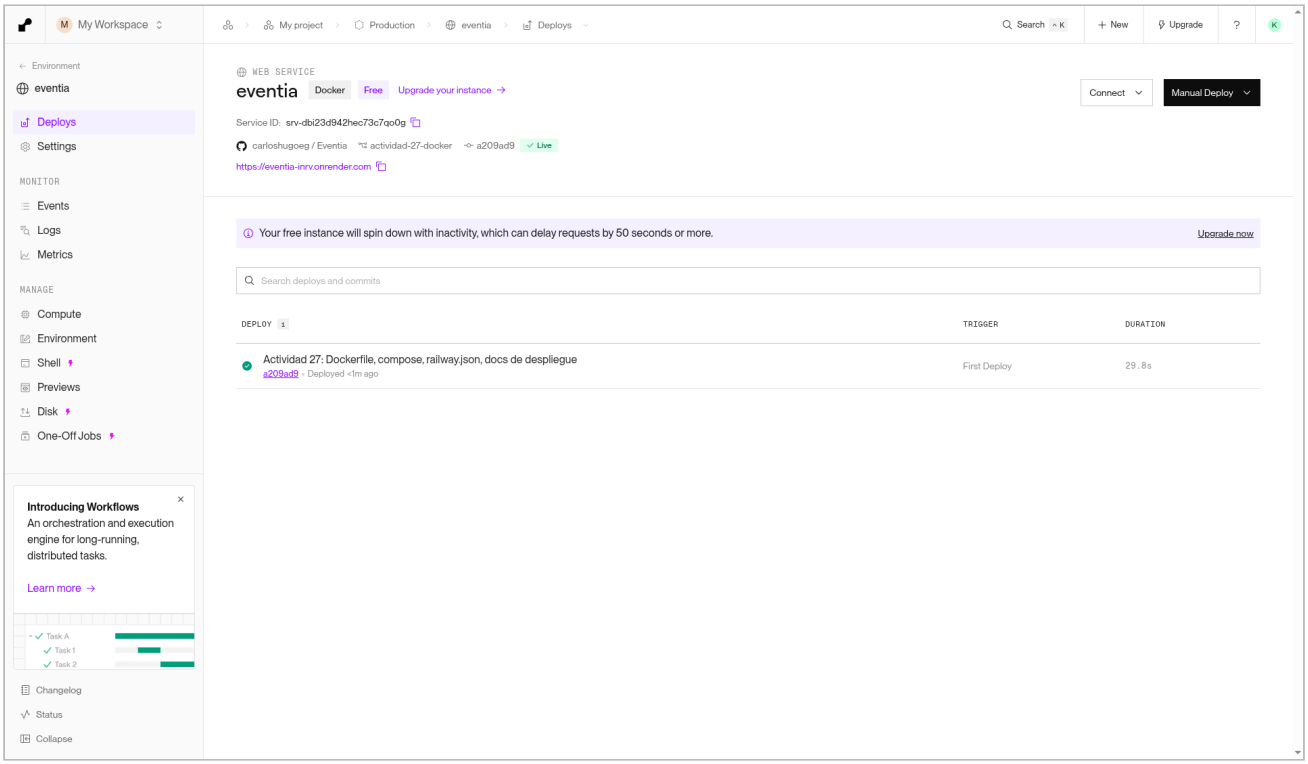


Figura 1. Servicio Docker en Render (free, Oregon) en estado Live

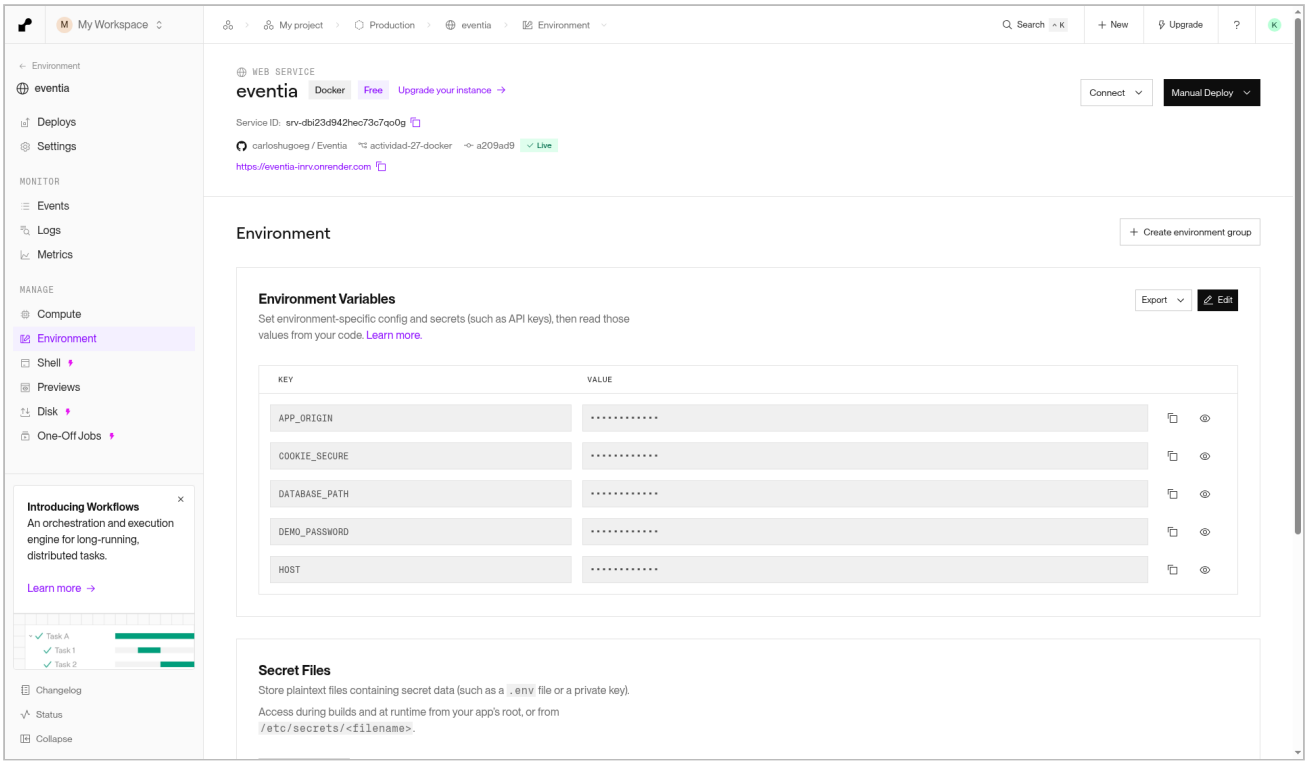


Figura 2. Variables de entorno configuradas en Render

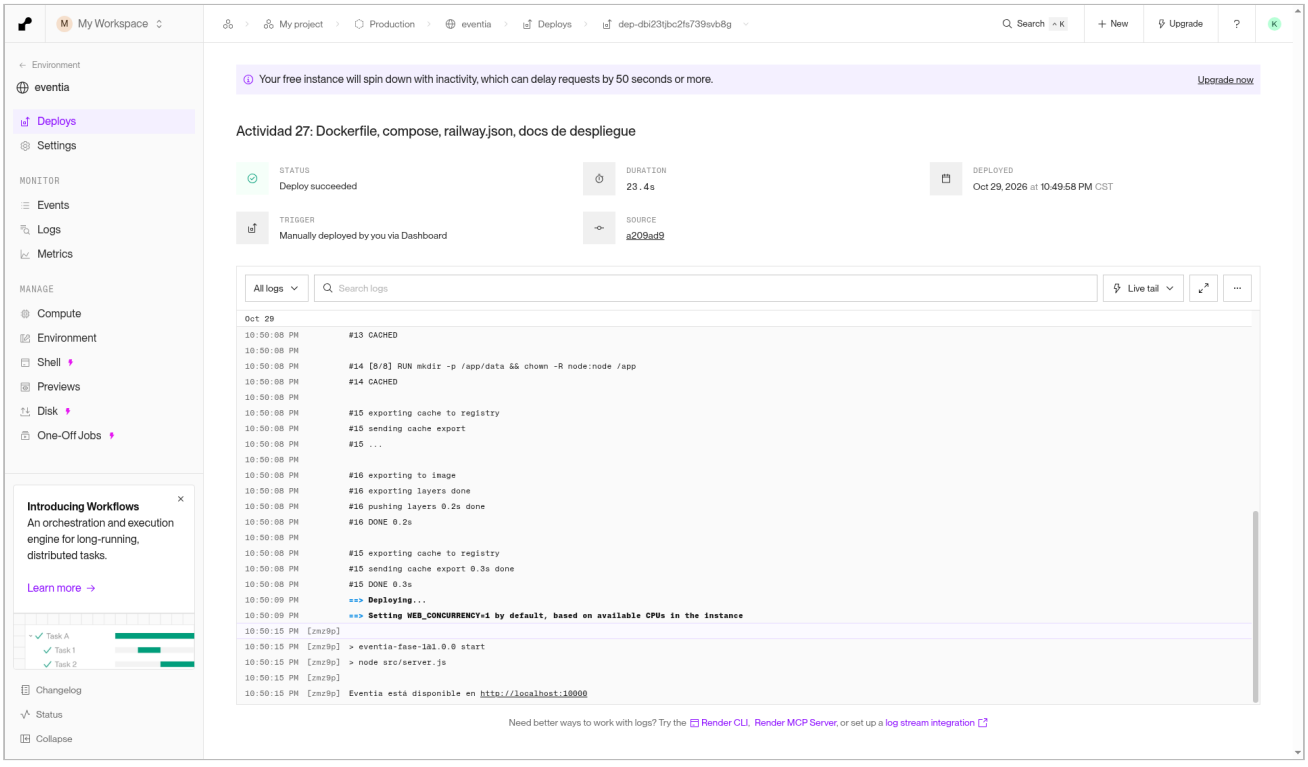


Figura 3. Logs de despliegue en Render



Figura 4. Página de inicio pública de Eventia

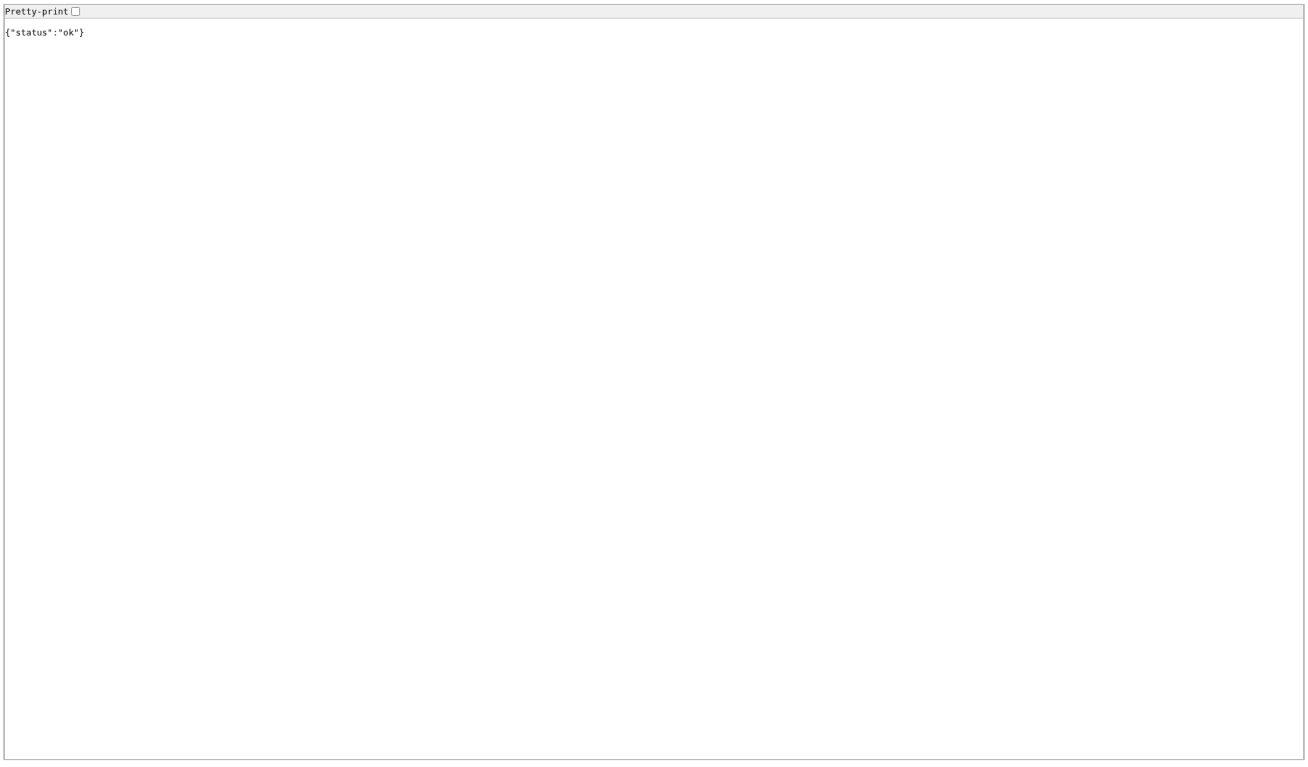


Figura 5. Health check /api/health → {"status":"ok"}

3. Docker

Dockerfile

```
FROM node:24-slim

ENV NODE_ENV=production \
    HOST=0.0.0.0 \
    PORT=3000 \
    DATABASE_PATH=/app/data/eventia.sqlite

WORKDIR /app

COPY package.json ./
COPY src ./src
COPY public ./public
COPY database ./database
COPY scripts ./scripts

RUN mkdir -p /app/data && chown -R node:node /app

USER node
VOLUME /app/data
EXPOSE 3000

CMD ["npm", "start"]
```

docker-compose.yml

```
services:
  eventia:
    build: .
    image: eventia:local
    ports:
      - "${PORT:-3000}:3000"
    environment:
      HOST: 0.0.0.0
      PORT: 3000
      DATABASE_PATH: /app/data/eventia.sqlite
      APP_ORIGIN: ${APP_ORIGIN:-http://localhost:3000}
      COOKIE_SECURE: ${COOKIE_SECURE:-false}
      DEMO_PASSWORD: ${DEMO_PASSWORD:-Eventia2026!}
    volumes:
      - eventia-data:/app/data
    restart: unless-stopped

volumes:
  eventia-data:
```

Comandos documentados (README – Despliegue y Docker)

Docker:

```
docker build -t eventia:local .
docker volume create eventia-data
docker run -d --name eventia -p 3000:3000 \
  -e APP_ORIGIN=http://localhost:3000 \
  -e DEMO_PASSWORD='Eventia2026!' \
  -v eventia-data:/app/data eventia:local
curl http://localhost:3000/api/health
# Datos de demostración (opcional):
docker exec eventia npm run seed
```

Prueba de persistencia:

```
docker rm -f eventia
docker run -d --name eventia -p 3000:3000 -v eventia-data:/app/data eventia:local
curl http://localhost:3000/api/events   # los datos creados siguen ahí
```

Docker Compose:

```
docker compose up -d --build
docker compose exec eventia npm run seed
docker compose logs -f eventia
docker compose down          # conserva el volumen eventia-data
docker compose down -v       # elimina también los datos
```

Pruebas automatizadas (npm test)

Resultado: **22 pruebas, 22 aprobadas, 0 fallidas** (node --test, ~2.1 s). Incluye flujo completo de inscripción, autenticación, sobrecupo, concurrencia (8 conexiones por 3 cupos) y validaciones HTTP.

Construcción de la imagen

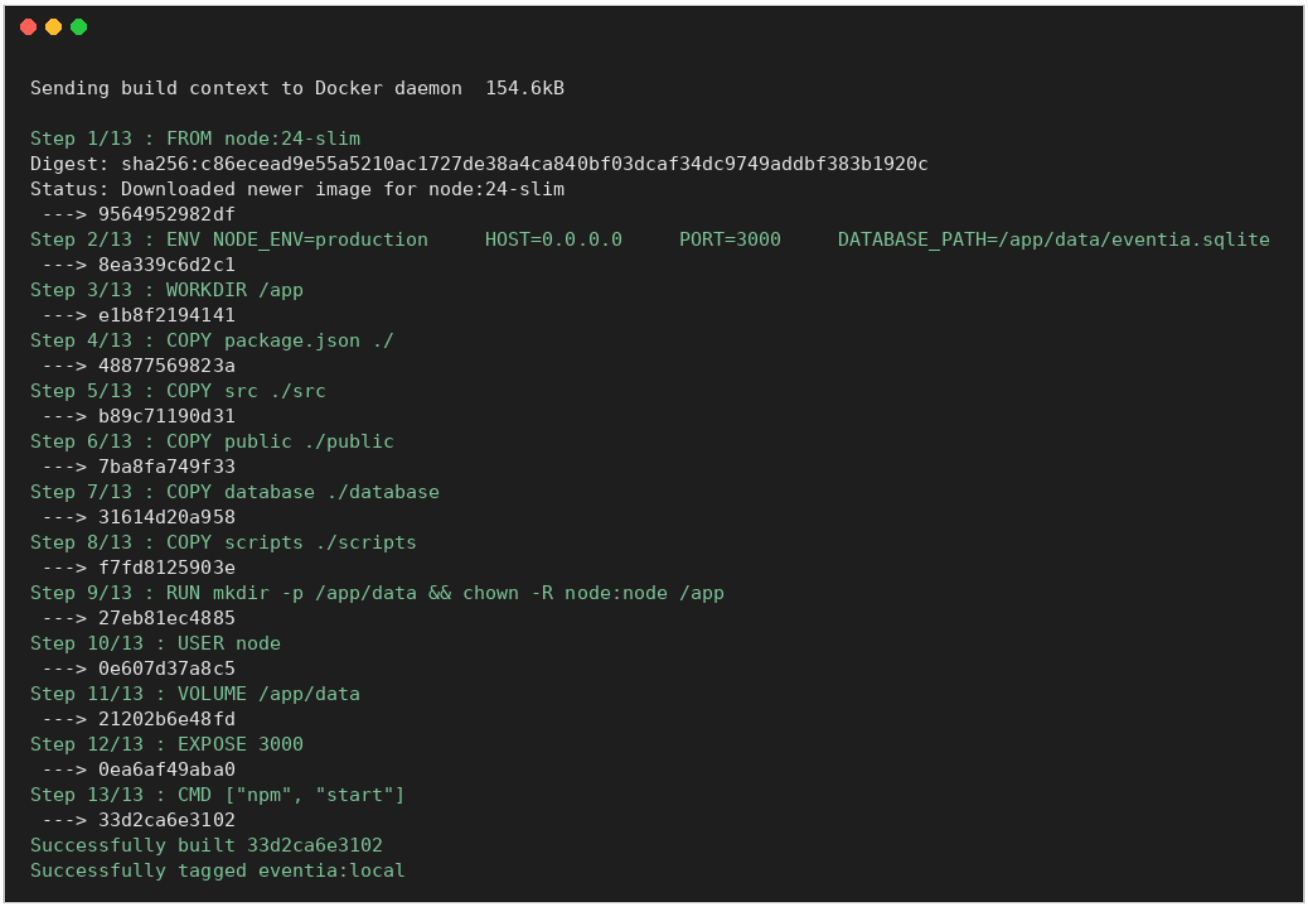


Figura 6. docker build -t eventia:local . (salida recortada: se omiten líneas de descarga de capas)

4. Evidencia de persistencia

1. Se creó el volumen nombrado `eventia-data` y se inició el contenedor montándolo en `/app/data`.
2. Se registró un organizador y se creó el evento «Evento persistente Docker»; la base SQLite quedó en `/app/data/eventia.sqlite`.
3. Se eliminó el contenedor con `docker rm -f eventia`.
4. Se creó un contenedor nuevo con el mismo volumen: el evento sigue apareciendo (`total: 1`) y el login del organizador funciona.

```
+ D='sudo docker'
+ O=http://localhost:3100
+ sudo docker rm -f eventia
+ sudo docker volume rm eventia-data
+ sudo docker volume create eventia-data
eventia-data
+ run
+ sudo docker run -d --name eventia --network host -e PORT=3100 -e APP_ORIGIN=http://localhost:3100 -v eventia-data:/ap…
9ed2446794d2a769682e5a8431ff263dc60142fbd34d394f855327ff525968dc
+ sleep 3
+ sudo docker ps --filter name=eventia
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS     NAMES
9ed2446794d2   eventia:local   "docker-entrypoint.s…"   3 seconds ago   Up 3 seconds             eventia
+ curl -s http://localhost:3100/api/health
{"status":"ok"}+ echo
+ curl -s -c /tmp/cj -H 'Origin: http://localhost:3100' -H 'Content-Type: application/json' -d '{"name":"Org Docker","e…
{"user":{"id":"10790803-30ef-40a6-a326-0c125d5b05b8","name":"Org Docker","email":"org-docker@example.com","role":"organ…
+ curl -s -b /tmp/cj -H 'Origin: http://localhost:3100' -H 'Content-Type: application/json' -d '{"title":"Evento persis…
{"event":{"id":"2f2834cc-cd82-4aa4-b576-ebf5883b6697","organizer_id":"10790803-30ef-40a6-a326-0c125d5b05b8","title":"Ev…
+ curl -s 'http://localhost:3100/api/events?q=persistente'
{"events":[{"id":"2f2834cc-cd82-4aa4-b576-ebf5883b6697","organizer_id":"10790803-30ef-40a6-a326-0c125d5b05b8","title":"…
+ sudo docker exec eventia ls -la /app/data
total 188
drwxr-xr-x 2 node node   4096 Oct 30 04:43 .
drwxr-xr-x 1 node node   4096 Oct 30 04:43 ..
-rw-r--r-- 1 node node   4096 Oct 30 04:43 eventia.sqlite
-rw-r--r-- 1 node node  32768 Oct 30 04:43 eventia.sqlite-shm
-rw-r--r-- 1 node node 144232 Oct 30 04:43 eventia.sqlite-wal
+ sudo docker rm -f eventia
eventia
+ sudo docker ps -a --filter name=eventia
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
+ run
+ sudo docker run -d --name eventia --network host -e PORT=3100 -e APP_ORIGIN=http://localhost:3100 -v eventia-data:/ap…
b30a9638d0e7366f7f837b605a1ec8a4367f53ada3739a30dfcc1591faa53d30
+ sleep 3
+ curl -s 'http://localhost:3100/api/events?q=persistente'
{"events":[{"id":"2f2834cc-cd82-4aa4-b576-ebf5883b6697","organizer_id":"10790803-30ef-40a6-a326-0c125d5b05b8","title":"…
+ curl -s -H 'Origin: http://localhost:3100' -H 'Content-Type: application/json' -d '{"email":"org-docker@example.com",…
{"user":{"id":"10790803-30ef-40a6-a326-0c125d5b05b8","name":"Org Docker","email":"org-docker@example.com","role":"organ…
+ sudo docker volume inspect eventia-data
[
    {
        "CreatedAt": "2026-10-29T22:43:25-06:00",
        "Mountpoint": "/var/lib/docker/volumes/eventia-data/_data",
        "Name": "eventia-data",
    }
]
+ sudo docker logs eventia
> eventia-fase-1@1.0.0 start
> node src/server.js
Eventia está disponible en http://localhost:3100
+ sudo docker rm -f eventia
eventia
```

Figura 7. Prueba de persistencia con volumen Docker (local; líneas largas recortadas)

> **Nota:** el plan free de Render no incluye disco persistente, por lo que la persistencia en la nube no está garantizada (los datos pueden perderse en reinicios o redeploys). La evidencia de volumen persistente corresponde a Docker local.

5. Diagrama de arquitectura

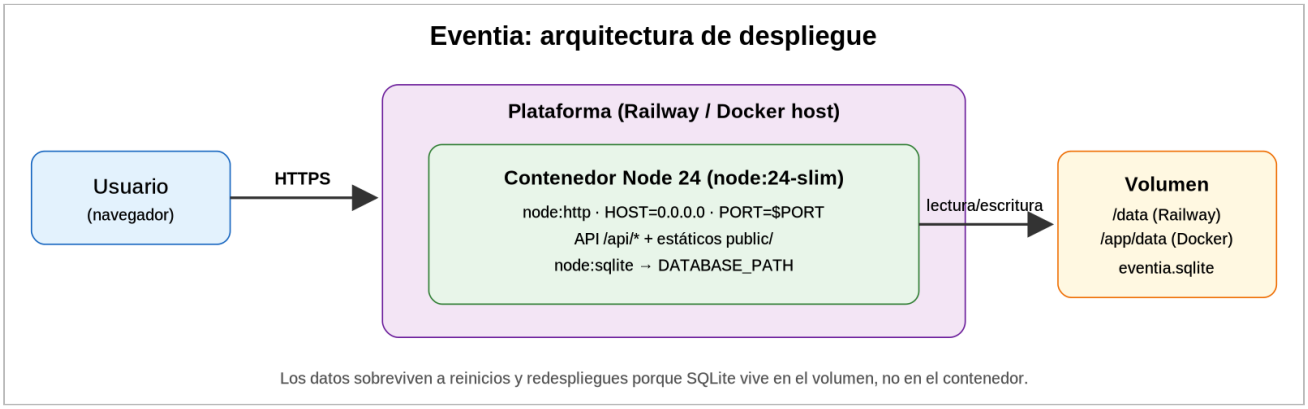


Figura 8. Arquitectura de despliegue de Eventia

6. Análisis técnico

1. ¿Qué plataforma se utilizó y por qué?

Nuestra primera opción era usar Railway, pero hoy en la noche tenían cerrados los registros, así que nos terminamos pasando a Render. La verdad es que Render nos funcionó súper bien porque construye el proyecto directamente desde el Dockerfile, nos da una URL con HTTPS gratis y se conecta directo a nuestro repositorio público sin complicaciones.

2. ¿Qué componentes se desplegaron y dónde?

Tenemos todo corriendo en un solo contenedor con Node 24, que se encarga de servir tanto la API como el frontend estático, y ahí mismo adentro tenemos SQLite para la base de datos. Todo esto está alojado en Render, en la región de Oregon, corriendo en su URL pública, y le configuramos un health check. Para trabajar en local, lo levantamos usando Docker Compose y los datos los guardamos en un volumen que llamamos eventia-data.

3. ¿Qué dificultades encontraron?

Nos topamos con tres problemas principales durante el proceso. Primero, los registros de Railway estaban cerrados, así que no pudimos crear la cuenta que queríamos; lo solucionamos migrando el despliegue a Render. Segundo, el plan gratuito de Render no tiene disco persistente, por lo que la base de datos de SQLite se borra si el contenedor se reinicia; lo aceptamos así por ahora porque es solo una prueba. Tercero, el cold start es de unos 50 segundos porque Render duerme la aplicación cuando no hay tráfico; también tuvimos que aceptarlo como una limitación de usar un servicio gratuito.

4. ¿Qué ventajas aporta Docker?

Lo mejor de usar Docker es que esta misma imagen corre exactamente igual en nuestra máquina que en la nube; no hay sorpresas. En local, la persistencia de los datos funciona gracias al volumen que configuramos. De hecho, hicimos la prueba de borrar el contenedor a propósito y la data siguió intacta. Además, todo el entorno se levanta con un solo comando y lo configuramos para que corra de forma segura con un usuario sin permisos root.

5. ¿Qué mejoras harían para producción?

Si quisiéramos llevar este proyecto a un entorno de producción real, le agregaríamos un disco persistente o un Postgres manejado para que la base de datos no se pierda cada vez que se reinicie el contenedor. También usaríamos un plan de pago sin sleep mode para que la app responda siempre al instante. Configuraríamos Integración Continua (CI) para automatizar la ejecución de los 22 tests antes de cada despliegue y asegurar que no subimos código roto. Finalmente, agregaríamos backups y monitoreo de salud, porque necesitamos respaldos por si ocurre un desastre y alertas por si el servicio se llega a caer.